

Server Routes

Differences with PWA Kit

PWA Kit server routes are typically centralized as Express handlers in SSR server code. Storefront Next replaces that with file-based resource routes and loader/action handlers using standard Web Request / Response APIs. This distributes server behavior across route modules and aligns API endpoints with routing conventions.

Contents

Key Differences

- PWA Kit Express Routes
- Storefront Next Resource Routes
- Key Similarities
- Storefront Conversion Considerations

Key Differences

Aspect	PWA Kit	Storefront Next
Pattern	Express routes in <code>ssr.js</code>	File-based resource ro
Configuration	Centralized in one file	Distributed across rou
Request API	Express <code>req / res</code> objects	Web Standard Request / Response
Route Definition	<code>app.get('/path', handler)</code>	Export <code>loader / acti</code> file
Dynamic Routes	Express params (<code>/:param</code>)	File naming (<code>\$param</code>)
Middleware	Express middleware chain	React Router middlew
Client Execution	Server only	Optional <code>clientLoader / cli</code>

PWA Kit Express Routes

```
// Single file: app/ssr.js
app.get("/api/search", async (req, res) => {
  const query = req.query.q;
  const results = await searchProducts(query);
  res.json(results);
});
```



Storefront Next Resource Routes

```
1 // src/routes/resource.search.ts
2 export function loader({ request }: LoaderFunctionArgs) {
3   const url = new URL(request.url);
4   const query = url.searchParams.get("q");
5   const results = await searchProducts(query);
6   return Response.json(results);
7 }
8
9 // src/routes/resource.auth.login.ts
10 export async function action({ request }: ActionFunctionArgs) {
11   const { email, password } = await request.json();
12   const session = await login(email, password);
13   return Response.json(session);
14 }
```

Key Similarities

- Both support GET (loaders) and POST/PUT/DELETE (actions).
- Both can set cache headers and response status codes.
- Both integrate with SLAS for authentication callbacks.
- Both can proxy Commerce API calls.
- Both support dynamic route parameters.

Storefront Conversion Considerations

Keep these tips in mind when you convert your PWA Kit storefront to Storefront Next.

- Route extraction: Move logic from `ssr.js` callbacks into separate route files.
- Request handling: Convert Express `req.body` / `req.query` to `request.json()` / `URL.searchParams`.
- Response handling: Convert `res.json()` to `Response.json()`.
- Middleware: Use React Router middleware exports instead of Express middleware.
- Client-side option: Consider adding `clientLoader` for endpoints that can run on the client.

DID THIS ARTICLE SOLVE YOUR
ISSUE?

[Share your feedback](#)



Try for free



DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblazer Community
Events and Calendar
Partner Community
Blog
Salesforce Admins
Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved.](#) Various trademarks held by their respective owners. Salesforce, Inc.
Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)